

ARCHITECTURE REVIEW

hillclimb vs. the autoresearch landscape

How the search policy, operators, evaluator, memory and harness are factored in 20 systems, where hillclimb stands, and what to improve

Sebastian Haglund El Gaidi · rebase.energy · 22 August 2026

Sources: awesome-auto-hillclimb, project source code and papers (see last slide)

What this deck answers

Three questions, one vocabulary

1. How do others structure the loop?

Where the "what next" decision lives (search policy), what the LLM subroutines are (operators), how a candidate becomes a number (evaluator), and what the system remembers (memory).

2. Where is hillclimb ahead or behind?

Pros and cons against the strongest open systems in each family.

3. What should change?

Concrete, prioritised improvement points, each traced to a system that already does it.

Systems surveyed (from awesome-auto-hillclimb)

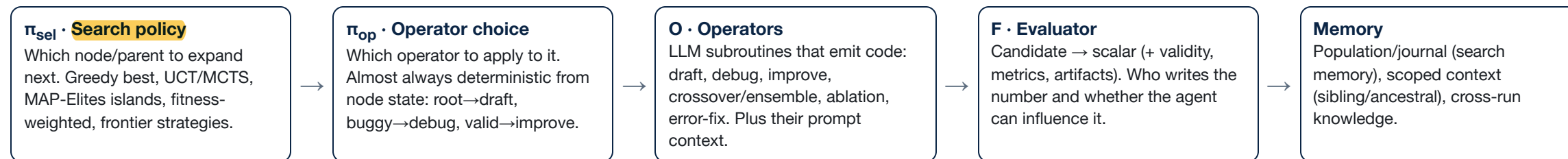
Family	Systems
Autoresearch loops	karpathy/autoresearch, pi-autoresearch, evo, CORAL, AutoKernel
Evolutionary program search	AlphaEvolve / FunSearch (reference), OpenEvolve, ShinkaEvolve, CodeEvolve, FM Agent, PIKE
ML-engineering agents	AIDE, weco-cli, AIRA-dojo, R&D-Agent, MLE-STAR, MLEvolve, ML-Master

Two empirical anchors the list leads with

- **Operators beat policy** (AIRA, arXiv:2507.02554): swapping greedy for MCTS/evo with fixed operators moved little; better operators drove the gains.
- **Exploit + repair beats explore-heavy evolution** under real budgets (PIKE, MLSys 2026).
- **The evaluator is the weakest link**: 16% of audited agent tasks are hackable from the description alone (arXiv:2606.08960).

A common vocabulary

AIRA-dojo formalises every system here as the tuple $(F, \pi_{\text{sel}}, O, \pi_{\text{op}}, \tau)$



Where the seam is drawn differs a lot

- **Seam in code** (named policy class / module): AIRA-dojo `Solver`, R&D-Agent `Scheduler + Router`, evo `frontier_strategies.py`, PIKE `query_strategies.py`, **hillclimb** `SearchPolicy`.
- **Seam inside the database**: OpenEvolve, ShinkaEvolve, CodeEvolve, FunSearch (the `ProgramDatabase` owns sampling, islands, archive).
- **Fused into one agent class**: AIDE, ML-Master, MLEvolve.
- **No engine at all, the loop is prose**: karpathy/autoresearch, pi-autoresearch, CORAL (agent decides what to try; tools enforce keep/revert).

Harness responsibilities nobody calls "policy"

- Candidate dirs / worktrees, journal writes, checkpoints, resume
- Budget (time, steps, cost), timeouts, parallel slots
- Accept/keep decision (greedy keep-if-better, MAP-Elites cell replacement, LLM-judged accept in R&D-Agent)
- Holdout / gates / private evaluation

hillclimb draws the line explicitly: "WHAT to try next is a policy decision; everything else is the harness." (`policy.py` docstring)

hillclimb at a glance

Run → Search → Candidate → Trial; one engine process per search; headless Claude Code agents as operators; the verifier is the problem

Harness · GreedySearcher

`search.py` : thread-pool of operator slots, three-phase candidate lifecycle (prepare → execute → commit), single writer of `journal.jsonl` / `status.json` / `best/`, control queue, wall-clock + USD budget, SIGTERM drain.

Policy · SearchPolicy

`propose(view) → Action`, `observe(view, candidate)`. Replay-deterministic by contract. `greedy` : debug buggy tip → ensemble window → draft xN → improve best unbusy. `openevolve` : MAP-Elites islands, parent + inspirations, seeded RNG.

Operators · prompts + routing

baseline, draft, debug, improve, ensemble (+ seed, distill, consolidate). Retrieval draft and ablation improve are prompt scaffolds. Model routing per operator with a UCB1 bandit over a model pool; reward gated by the accept band.

Backend · OperatorBackend

`claude -p` headless session per call, cwd = candidate dir, stream-json transcript, rate-limit → park. Dummy/fake backends for tests.

Evaluator · verifier.sh

Only process the engine starts. Writes `$HILLCLIMB_RESULT` (score + numeric metrics → `Trial.metrics`), exit 0 = valid. Deleted before each run; never read off stdout. `--holdout` in a dir agents never see. Providers: `emflow://`, `mlebench://`.

Noise model

Score = median of `n_trials`; spread = MAD; accept band = $\max(\text{min_improvement}, \text{noise_k} \times \text{measured floor})$; `trial_mode: serial` for machine metrics; `hillclimb verify --repeat` prints the floor.

Knowledge · knowledge/

Cards → typed claims (helps/hurts/...) → derived graph with credit-adjusted confidence → playbooks, skills library (reference solutions). Live sibling cards shared across parallel searches. Retrieval into draft/improve prompts; agents can query mid-search.

Observability

`watch` (agents), `chart` (score vs time), `graph` (3D knowledge graph), `tree` (png), `demo`.

Selection for `best/`: `rank-blend` (min of val rank + holdout rank) by default, so an overfit validation score is vetoed by holdout. Holdout scores never enter prompts; MLE-bench grading runs once, after the search.

How each system factors policy vs operator

Where the "what next" decision lives, and what the operators are

System	Search policy (π_{sel})	Operators (O) and π_{op}	Policy/operator seam
AIDE	Greedy: draft until N, then p=0.5 debug a buggy leaf, else improve best node	draft / debug / improve, chosen by node state; LLM "feedback" parses metric	Fused in <code>Agent</code> ; <code>search_policy()</code> is a method
AIRA-dojo	Greedy, MCTS (UCT, 5 children, no rollouts), Evo (fitness-proportional, crossover) as swappable <code>Solver</code> s	draft/debug/improve/analyze/crossover/memory as plain functions bound as partials; deterministic by node state	Clean: one file per policy, operators in <code>operators/</code> , Hydra configs
R&D-Agent	<code>Scheduler</code> (round-robin, SOTA-based, MCTS/PUCT) picks trace/parent; <code>Router</code> picks draft/idea/merge	Researcher (hypothesis) vs Developer (six CoSTEER coders, each an inner evolve loop)	Clean two-level split; but large, scenario-coupled
MLE-STAR	None (fixed schedule): seed $\rightarrow$ T $\times$ K ablation-guided refine $\rightarrow$ ensemble	~10 named agents: retriever, ablation, extractor, coder, planner, ensembler, leakage/data checkers	No policy layer
MLEvolve	Monte-Carlo <i>graph</i> search: UCT with decaying c(t), stagnation triggers $\rightarrow$ fusion / aggregation / new root	9 agents: draft, improve (diff), debug, evolution, fusion, aggregation, code review, leakage, result parse	Fused; policy is UCT + stagnation state machine
ML-Master	MCTS with shaped reward; 3 async workers claim unoccupied UCT children	draft/debug/improve; memory spliced into the <code><think></code> block	Fused
OpenEvolve	Per-island MAP-Elites; explore 0.2 / archive 0.5 / fitness-weighted; ring migration	diff or full-rewrite prompt via <code>PromptSampler</code> ; inspirations + artifacts; weighted LLM ensemble	Policy = <code>ProgramDatabase</code> (2.6k lines); context assembly split between worker and sampler
ShinkaEvolve	Islands + weighted/power-law parent sampling + archive; novelty rejection before eval	diff / full / cross patch types; bandit (UCB/Thompson) picks the LLM; meta-summarizer	Policy inside DB + runner; no abstraction
CodeEvolve	CVT-MAP-Elites per island, tournament/roulette, scheduled exploration rate	diff inside EVOLVE-BLOCK; lineage-as-chat; co-evolved prompts (meta-prompting)	<code>EliteMap</code> ABC (grid vs CVT) is the cleanest QD abstraction
PIKE	100% exploit: top-4 of last phase, branch $\times 10$, library cleared each phase	idea $\rightarrow$ generate $\rightarrow$ error-fixing agent (≤ 5 rounds) $\rightarrow$ improve; always full rewrite	<code>query_strategies.py</code> named, only one wired
evo	<code>frontier_strategies.py</code> : argmax, top-k, ϵ -greedy, softmax, pareto-per-task	Any host coding agent in a git worktree; briefs written by orchestrator agent	Selection in code, loop in prose
pi-autoresearch / karpathy / CORAL	Agent decides (single lineage greedy; CORAL: decentralised multi-agent)	The coding agent itself; tools enforce keep/revert (pi), heartbeats steer (CORAL)	No engine-side policy
hillclimb	SearchPolicy protocol: greedy (AIDE-shaped + ensemble window) or OpenEvolve MAP-Elites, chosen per search	draft/debug/improve/ensemble prompts + UCB1 model routing; Claude Code agent per call	Explicit seam with replay-determinism contract; policy also picks the operator

Deep dive: AIDE → AIRA-dojo

The lineage hillclimb's greedy policy descends from, and the study that says what matters

AIDE's loop (≈ hillclimb greedy)

- `parent = search_policy(): None → draft; parent.is_buggy → debug; else improve`. Defaults `num_drafts=5`, `debug_prob=0.5`, `max_debug_depth=3`.
- Score: agent prints its own 5-fold CV metric; a second LLM parses stdout into `{is_bug, metric, summary}`.
- Memory: one global text summary of all good nodes in every prompt → mode collapse.
- No parallelism, no verifier, no holdout.

What AIRA changed, and the result

- Same greedy rule, same CV. Only operators changed: **scoped memory** (draft/improve see sibling summaries only; debug sees the ancestral chain), **complexity-adaptive prompts** (minimal/moderate/advanced), thinking tokens.
- MLE-bench Lite medal rate: AIDE-greedy 39.8% → AIRA-greedy 45.5% → AIRA-MCTS 47%.
- With AIDE operators, MCTS/evo gained *nothing*. "The operator set is the bottleneck."
- Validation→test gap of 9–13 points: selecting by val score is the largest remaining loss.

How hillclimb compares

Aspect	AIRA-dojo	hillclimb
Policy seam	<code>Solver.__call__</code> , checkpoint/load	<code>SearchPolicy.propose/observe</code> , journal replay
Operator choice	Deterministic from node state	Policy-owned (greedy mirrors AIDE; debug tip first)
Scoped memory	sibling / ancestral processors	Sibling summaries in improve, debug chain in debug, live cards ✓
Complexity cues	by child count	by draft index (<code>COMPLEXITY_CUES</code>) ✓
Score source	LLM parses stdout	verifier writes result file ✓
Policies shipped	greedy, MCTS, evo	greedy, openevolve (no tree-search policy)
Parallelism	across seeds only	N operators per search + N searches per run ✓
Val→test gap	measured, unaddressed	holdout + rank-blend selection ✓

Takeaway. hillclimb already implements the three AIRA operator upgrades and fixes the two AIDE weaknesses (stdout scoring, global memory). Its missing piece from this lineage is an MCTS/UCT policy, which AIRA shows is only worth it once operators are strong.

Deep dive: the AlphaEvolve family

OpenEvolve, ShinkaEvolve, CodeEvolve: the database is the policy, the prompt sampler is the operator

Shared reference design (AlphaEvolve / FunSearch)

- Prompt sampler draws parent + inspirations from a program DB → LLM ensemble emits SEARCH/REPLACE diffs or full rewrites inside `# EVOLVE-BLOCK` → cascade evaluator → DB insert. Islands with migration/reset.
- Ablations: evolution, rich context, meta-prompts, full-file evolution and strong models each matter.

Where the three open reproductions differ

	OpenEvolve	ShinkaEvolve	CodeEvolve
Archive	grid MAP-Elites per island, elite archive 100	single score + archive 40, novelty gate (embeddings + LLM judge)	CVT-MAP-Elites (<code>EliteMap</code> ABC)
Parent	explore 0.2 / archive 0.5 / fitness-weighted	weighted or power-law, sigmoid(F) / (1+children)	tournament/roulette, scheduled explore rate
Operator	diff or full rewrite; inspirations + artifacts	diff / full / cross; bandit picks LLM; meta-summarizer	diff; lineage-as-chat; co-evolved prompts
Evaluator	<code>evaluate(path) → dict</code> , cascade stages	<code>metrics.json</code> with public/private/text_feedback	<code>python evaluate.py code out.json</code>
Parallel	process pool, whole-DB pickle per task	asyncio + Slurm scheduler	process per island, barrier migration
Resume	JSON checkpoint dir	SQLite	pickle

hillclimb's position

- Wraps OpenEvolve's `ProgramDatabase` as a *policy only* (`policies/openevolve.py`): islands, MAP-Elites cells, inspirations, migration. Everything else (agents, verifier, journal, trials, holdout) stays hillclimb's.
- Feature dimensions come from `Trial.metrics` (numeric keys the verifier writes next to `score`), so QD descriptors are verifier-defined, not agent-defined.
- Seeds/restores the global `random` state around every OpenEvolve call → replay-deterministic, which OpenEvolve itself is not.
- Operator is a full agent session (reads the dir, runs code) rather than a one-shot diff; per-candidate cost is 10–100× an OpenEvolve sample.

Gaps vs this family. No novelty rejection before paying for an evaluation (Shinka); no cost-aware bandit over models (Shinka's AsymmetricUCB; hillclimb's UCB1 is reward-only); no crossover/inspiration-aware *prompt* beyond the rendered cell table; no meta-prompt / system-prompt evolution; no cascade evaluation.

Edges. Noise model (median + band) that none of the three have; verifier-owned score; holdout; cross-run knowledge; PIKE-style findings (1 island, exploit-heavy) are one `policy_params` line away.

Deep dive: R&D-Agent, MLEvolve, MLE-STAR, ML-Master

The MLE-bench leaders: richer operator sets, graph/tree policies, and scoped memory

R&D-Agent (Microsoft) · 35% medals, full MLE-bench

- **Policy in two layers:** `Scheduler` (round-robin, SOTA-based, MCTS-PUCT) picks which trace/parent; `Router` picks draft / idea / merge.
- **Operators:** Researcher proposes a hypothesis from performance feedback; Developer = six CoSTEER coders (data loader, feature, model, ensemble, workflow, pipeline), each an inner evolve→evaluate loop with its own RAG of past implementations.
- Multi-trace parallel search merged at the end; LLM-judged accept that is "cautious of high valid scores".

MLEvolve (InternScience) · 65% medals at 12h

- **Monte-Carlo graph search:** tree edges + *reference* edges for cross-branch context; UCT with decaying exploration; stagnation counters trigger intra-branch evolution, cross-branch **fusion**, or **aggregation** into a new root.
- Nine operators incl. pre-exec code review and leakage check; coder has rewrite / stepwise / diff modes.
- Retrospective memory: BM25 + FAISS fused by RRF, stage-aware (plan text vs error text). Ablation: −13 pts without memory, −13 pts without graph search.

MLE-STAR (Google) · 64% medals, Lite

- No search tree. Web-search-seeded drafts merged greedily; then an **ablation agent** finds the highest-impact code block and an inner loop edits *only that block*; ensemble planner at the end.
- Guards: debugger, leakage detector, data-usage checker.
- Sharpest "improve" operator in the survey; fixed schedule cannot reallocate effort.

ML-Master (SJTU) · 29% medals, open-weight model

- MCTS with shaped reward (−1 on defect; improvement over best-so-far with threshold 0.001 + debug + structural terms); branch stops after 3 failed improves.
- Three async workers claim unoccupied UCT children; memory = parent insights + sibling alternatives spliced into the reasoning trace.

Relevance to hillclimb. hillclimb already has ablation-guided improve (`improve_ablation` , sibling `ablation.md`), sibling-scoped memory and an ensemble operator. It lacks: a tree/graph policy with stagnation-triggered recombination (MLEvolve), a Researcher/Developer split where hypothesis and code are separate journaled objects (R&D-Agent), pre-execution review and leakage/data-usage checkers as operators (MLE-STAR, MLEvolve), and a retrieval index over past candidates' plans and errors (MLEvolve).

Deep dive: agent-as-operator loops

karpathy/autoresearch, pi-autoresearch, evo, CORAL: the closest cousins in how the operator is run

	karpathy/autoresearch	pi-autoresearch	evo	CORAL	hillclimb
Loop owner	Agent reading <code>program.md</code> ; no engine	Agent + three tools (<code>init/run/log_experiment</code>) that enforce keep/revert	Orchestrator agent follows a skill; subagents per experiment	N long-lived agents, grader daemon, heartbeats (reflect/consolidate/pivot)	Engine process; agent is a stateless one-shot operator call
Policy	Single lineage greedy	Single lineage greedy	<code>frontier_strategies.py</code> (argmax, top-k, ϵ -greedy, softmax, pareto)	Decentralised; agents decide	<code>SearchPolicy</code> : greedy / MAP-Elites
Isolation	git branch, reset on fail	git commit/checkout	git worktree per experiment; remote pools (Modal, E2B, SSH...)	worktree per agent; Docker; separate grader venv + private dir	candidate dir per call; no git; machine-wide flock slots
Score	grep stdout	<code>METRIC name=value</code> on stdout; <code>checks.sh</code> gate	JSON to <code>\$EVO_RESULT_PATH</code> ; gates can veto a better score	<code>TaskGrader.evaluate()</code> in daemon, private dir agent cannot read	verifier writes <code>\$HILLCLIMB_RESULT</code> ; holdout dir
Noise	none (one 5-min run)	MAD confidence, advisory	none found	none	median of trials + accept band + measured floor
Memory	<code>results.tsv</code> + git log	<code>prompt.md</code> , <code>log.jsonl</code> with hypothesis/learnings, <code>ideas.md</code>	traces, annotations, discarded hypotheses readable by all agents	shared <code>.coral/public/</code> notes + skills; 36% of attempts reuse others' code	journal + live cards + knowledge graph/playbooks/skills across runs
Resume	re-read log	compaction-safe, log replay	on-disk tree	saved agent session ids	journal replay, policy/bandit rebuilt, budget restored

hillclimb is ahead on engine-owned acceptance (agents cannot self-report "keep"), noise handling, replay determinism, and cross-run knowledge.

hillclimb is behind on evaluator isolation (CORAL runs the grader in a separate venv/process with a private dir; hillclimb's verifier runs in the candidate dir with the agent's venv), first-class **gates** (evo), per-candidate git history (evo/pi make every kept change a reviewable commit), and remote execution pools (evo).

Evaluator contracts compared

Who writes the number, and what stands between the agent and its reward

Family	Contract	Systems	Agent can influence score?
LLM parses stdout	A second model reads logs → metric + bug flag	AIDE, AIRA-dojo, ML-Master, MLEvolve, R&D-Agent (partly)	Yes: prints the metric itself; known hallucination/leak vector
Regex on stdout	<code>metric: value</code> or <code>METRIC name=value</code>	weco-cli, karpathy, pi-autoresearch	Yes: same stream as agent code
Python <code>evaluate()</code> → dict	importlib-loaded function returns <code>combined_score</code> + metrics (+ cascade, artifacts)	OpenEvolve, FM Agent, PIKE-O, AlphaEvolve	Evaluator file fixed, but runs in-process with the program
Subprocess writes result file	JSON at a path the harness owns; exit code = validity	CodeEvolve, ShinkaEvolve (<code>metrics.json</code> + <code>correct.json</code>), <code>evo (\$EVO_RESULT_PATH + gates),</code> <code>hillclimb (\$HILLCLIMB_RESULT)</code>	Only via the solution; score deleted before each run
Grader daemon + private dir	Grader class in its own venv/process, reads data the agent cannot	CORAL; hillclimb <code>--holdout</code> (partially)	No; agent never sees grader inputs

Extras worth noting

- **Shinka** splits metrics into `public` (shown to LLM), `private`, `text_feedback`. hillclimb has `report` (validation only) and `Trial.metrics`, but no explicit public/private split on the validation side.
- **OpenEvolve/AlphaEvolve** cascade: cheap stage-1 test gates expensive stage-3. hillclimb runs the full verifier every time.
- **evo** gates: exit-0 scripts that veto a candidate regardless of score; `discover` requires a held-out score-floor gate.
- **PIKE**: async parallel evaluation biased toward fast-to-evaluate low-quality programs; hillclimb's `trial_mode: serial` is the right fix.

hillclimb's evaluator is the strongest open one in the list, except CORAL's isolation

- Verifier is the only process the engine starts; score file deleted pre-run; NaN rejected; exit 0 without file = contract violation
- Holdout split in a directory agents never see; holdout reports hard-rejected from prompts; top-k gating for query hygiene
- Credential scrubbing for agent-authored code
- Self-reported metrics (MLE-bench) explicitly labelled `report_trusted=False`
- Noise floor measured and enforced (no other system does both)

Remaining exposure

- Verifier runs with the candidate's venv in the candidate dir → a solution can monkey-patch imports the verifier uses
- No adversarial hardening loop (hacker/fixer, arXiv:2606.08960) for problem authors
- No gates separate from score (e.g. "must use all data files", "no test-statistics in preprocessing")
- No isomorphic-perturbation re-scoring of the winner

Memory and knowledge compared

Three layers: search memory, scoped prompt context, cross-run knowledge

System	Search memory (within a run)	Context given to an operator	Cross-run knowledge
AIDE	journal.json (nodes)	Global summary of all good nodes (causes mode collapse)	none
AIRA-dojo	journal.jsonl	Scoped: sibling summaries for draft/improve, ancestral chain for debug	none
R&D-Agent	Trace DAG of (experiment, feedback)	SOTA experiment + hypothesis history	CoSTEER RAG of past implementations (pickled)
MLEvolve	Graph nodes with plan/code/metrics/success	BM25 + FAISS retrieval, stage-aware (plan vs error), reference edges across branches	Curated cold-start KB by task type
OpenEvolve / CodeEvolve	Population + MAP-Elites grid + archive	Top programs, previous attempts, inspirations, artifacts (stderr); CodeEvolve: lineage-as-chat	none
ShinkaEvolve	SQLite (programs, archive, attempts, embeddings)	Archive + top-k inspirations, text_feedback, meta-summarizer recommendations	none
pi-autoresearch	log.jsonl with hypothesis/learnings/next-focus per run	prompt.md "what's been tried", ideas.md	prompt.md survives, manual
CORAL	attempts + notes + skills in .coral/public	Whatever the agent reads; heartbeats force reflect/consolidate	checkpointed public dir
hillclimb	journal.jsonl; live cards republished per candidate	Scoped: sibling summaries + ablations for improve, debug chain for debug, parent score delta, eval breakdown, live sibling cards, retrieved claims/playbook, reference solution	Cards → typed claims → graph with credit-adjusted confidence → playbooks + skills library; A/B bench of learning on/off

Unique to hillclimb: typed, credit-scored claims (helps/hurts/outperforms...) with retirement; credit assigned per search outcome; an A/B harness to measure whether knowledge injection helps at all.

Missing relative to others: retrieval over *raw* past candidates (plans, code, error text) within and across runs (MLEvolve, R&D-Agent); a running meta-summary the policy itself consumes (Shinka); per-candidate hypothesis/learning fields the agent fills in (pi's `asi`), which would make claims extraction cheaper and more reliable.

Parallelism, budgets and persistence

How runs scale out and come back

System	Parallelism	Budget	State / resume
AIDE / AIRA	none inside a search; seeds via Slurm	steps, 24h	journal JSON(L); reconstruct tree
R&D-Agent	multi-trace async, merge at end	time	pickled loop
ML-Master / MLEvolve	3 workers / 3 branches on one tree/graph	12h, 500 expansions	not described
OpenEvolve	process pool, DB snapshot per task	iterations	checkpoint dir, evolution trace
ShinkaEvolve	asyncio; local or Slurm jobs; 6 proposals / 4 evals	USD cap , generations	SQLite
CodeEvolve	process per island, barrier migration	epochs	pickle per island
FM Agent	Ray multi-node, separate gen/eval pools	iterations	server-side
PIKE	10 async workers, phases are barriers	query budget (300)	no resume
evo	worktrees; local pool, SSH, Modal, E2B, Daytona, AWS, Azure	stall counter	on-disk tree
CORAL	N agents async, islands in v0.6	time	agent session ids
hillclimb	N operators per search (thread pool) × N detached engines per run; machine-wide flock slots	wall-clock + margin, max_candidates, USD ceiling (parks)	append-only journal replay; policy + bandit rebuilt; budget restored; stale stops cleared; orphan reaper

Ahead

- Only system with cost ceiling *and* wall-clock *and* candidate cap, all resumable
- Replay determinism as a stated policy contract; no pickles
- Park-not-crash on rate limits; in-flight drain on stop
- Control queue keeps the engine the single writer

Behind

- Single machine only; no remote execution pool (evo, FM Agent, Shinka/Slurm)
- No per-LLM-call query budget (PIKE) or cost-aware routing (Shinka)
- Completion-order commit means parallel improves of the same parent race to the band (PIKE's fast-eval bias, in a milder form)
- No git history per candidate; diffs between candidates must be computed after the fact

Where hillclimb is ahead

Summary of pros against the field

1. The problem is its verifier

One contract (`verifier.sh` → `$HILLCLIMB_RESULT`, exit 0), providers supplying argv for the same contract, holdout by flag. Cleaner than OpenEvolve's Python function and safer than every stdout-based system. Only CORAL's isolation is stronger.

2. A real noise model

Median of trials, MAD spread, accept band from a measured floor, serial trials for machine metrics, band-gated bandit reward. No surveyed system combines these; most have none.

3. An explicit, replay-deterministic policy seam

`SearchPolicy` with a written contract; the same harness runs AIDE-style greedy and OpenEvolve MAP-Elites. AIRA-dojo is the only other system with swappable policies, and it has no verifier or parallelism.

4. Full coding agent as the operator

Agents read the data, run code, iterate inside one call; operators get structured context (score delta, eval breakdown, sibling ablations, debug chain). This is the lever AIRA and PIKE say matters most.

5. Cross-run knowledge with credit

Typed claims, confidence adjusted by downstream outcomes, retirement, playbooks, skills library, and an A/B bench to check it helps. Nothing else in the list closes this loop.

6. Operational maturity

Single-writer state, control queue, park/resume, cost ceiling, orphan reaping, live TUIs, demo. Comparable only to Shinka's WebUI/SQLite and evo's dashboard.

Where hillclimb is behind

Summary of cons, each with the system that does it better

1. Thin policy menu

Greedy and MAP-Elites only. No UCT/MCTS (AIRA, ML-Master), no graph search with stagnation-triggered fusion/aggregation (MLEvolve), no multi-trace merge (R&D-Agent), no scheduled explore→exploit (CodeEvolve). The greedy policy's operator choice is hard-coded, not configurable.

2. Operator set is the AIDE four

draft/debug/improve/ensemble. Missing: dedicated error-fix loop inside the operator (PIKE: +45% from this alone), crossover/fusion from other branches (MLEvolve, Shinka), pre-execution code review and leakage/data-usage checks (MLE-STAR, MLEvolve), hypothesis-first Researcher step (R&D-Agent), meta-prompt evolution (AlphaEvolve, CodeEvolve).

3. Expensive samples, no cheap filter

Every proposal is a full agent session and a full verifier run. No novelty rejection before evaluation (Shinka), no cascade evaluation (OpenEvolve/AlphaEvolve), no cost-aware model bandit.

4. Evaluator isolation

Verifier shares the candidate's venv and directory; no grader daemon / private dir for validation (CORAL); no gates beyond the score (evo); no adversarial hardening tool for problem authors.

5. Memory retrieval is claim-level only

No index over raw past candidates (plans, errors, code) for stage-aware retrieval (MLEvolve's BM25+FAISS, R&D-Agent's CoSTEER RAG). Agents never record a structured hypothesis/learning per candidate (pi's `asi`), so distillation relies on a separate LLM pass.

6. Scale-out and reviewability

Single machine; no remote pools (evo, FM Agent); no git worktree per candidate so kept changes are not reviewable commits (evo, pi, CORAL).

Improvement points: the policy layer

All implementable behind the existing `SearchPolicy` seam; none touch the harness

P1. Split π_{op} from π_{sel} inside greedy

Make operator choice a configurable rule table (`debug_prob` , `max_debug_depth` , improve-failure cap per branch as in ML-Master's $\tau_{improve}=3$) instead of the fixed priority list. Lets a branch die instead of re-improving a plateaued best.

P2. Add a `uct` policy

AIRA-MCTS: UCT over journal nodes, no rollouts, Q from the verifier score normalised by observed min/max, expand k children at once. Trivially replay-deterministic since Q/N are journal-derived. Worth it now that operators are strong (AIRA's condition).

P3. Stagnation triggers → recombination

MLEvolve's rule: after τ_{branch} failed improves on a node, propose `improve` with `inspiration_ids` from other branches (fusion); after τ_{global} , an `ensemble` -style aggregation into a new root. Needs only the existing `Action.inspiration_ids` .

P4. Exploit-heavy OpenEvolve preset

Ship PIKE's winning config as a named preset (`num_islands: 1` , `exploration_ratio: 0` , small archive, short library) and make it the default for timing metrics; keep islands for open-ended discovery problems.

P5. Novelty gate before spending an agent call

Shinka: embed the proposed parent+plan (or hash the solution) and reject near-duplicates before evaluation. Cheap win given each sample costs a full agent session. The greedy ensemble already md5-dedupes; generalise it.

P6. Multi-search merge

R&D-Agent's end-of-run merge: when a run holds several searches on one problem, schedule a final `ensemble` across searches' `best/` , scored on the same holdout. Today parallel searches only share live cards.

Improvement points: the operator layer

The lever with the best evidence behind it (AIRA, PIKE, MLE-STAR, ImprovEvolve)

O1. Error-fix loop inside the operator call

PIKE's single biggest gain. Today a buggy candidate costs a full round-trip (journal $\rightarrow$ policy $\rightarrow$ new debug candidate). Let the operator prompt run the verifier and fix up to N times before returning, journaling the attempts as `Trial`s rather than candidates. Keeps the journal clean and halves debug latency.

O2. Hypothesis as a first-class field

R&D-Agent / pi-autoresearch: require the agent to write `hypothesis.md` (what, why, expected delta) and `learnings.md` (what happened) into the candidate dir. Journal them; feed them to siblings and to claims distillation instead of re-deriving from transcripts.

O3. Crossover / fusion operator

A prompt that receives 2–3 solutions with their eval breakdowns and is asked to combine the strongest components (MLEvolve fusion, CodeEvolve inspiration crossover). Distinct from `ensemble` (which averages predictions).

O4. Verifier-side checker operators

MLE-STAR/MLEvolve guards as cheap pre-execution passes (haiku-class): data-leakage, all-data-used, "does this change match the hypothesis". Cheaper than a failed trial; also feeds the gates idea on the next slide.

O5. Cost-aware model bandit

Replace reward-only UCB1 with Shinka's AsymmetricUCB: reward = improvement magnitude ($\exp(\Delta)-1$) divided by cost, with decay. Pairs naturally with the existing per-operator routing and the USD ceiling.

O6. Meta-prompt evolution

AlphaEvolve/CodeEvolve: let the distill pass also propose edits to the operator prompt cues (e.g. `ablation_cue.md`) per problem family, credited through the same claims/credit machinery. Low priority until O1–O3 land.

Improvement points: evaluator integrity and cost

"Treat evaluator hardening as part of the loop, not a preliminary"

E1. Isolate the verifier from the candidate

CORAL pattern: run `verifier.sh` from a harness-owned copy in a separate venv, with the candidate mounted read-only and a `private/` dir the agent cannot list. Closes the import-shadowing hole and makes holdout and validation symmetric.

E2. Gates alongside the score

evo: `problems/<id>/gates/*.sh`, exit 0 = pass; a failing gate vetoes promotion even inside the accept band. Natural home for leakage and data-usage checks, runtime/memory ceilings, and "holdout $\geq$ floor" for discovery problems.

E3. hillclimb harden

Hacker/fixer loop from arXiv:2606.08960 as a CLI: one agent tries to max the score without solving the problem, another patches the verifier, a third confirms the baseline still passes. Run it when scaffolding a problem, same place `verify --repeat` lives.

E4. Cascade evaluation

OpenEvolve/AlphaEvolve: optional `verifier.sh --stage 1` (smoke, seconds) before the full run. Lets O1's fix loop iterate cheaply and saves full trials on obviously broken code.

E5. Public / private validation metrics

Shinka's split: verifier marks which `Trial.metrics` keys may enter prompts. Today all numeric keys are journaled and the report is validation-only; an explicit allow-list makes "what the agent can see" auditable.

E6. Winner re-scoring

Isomorphic-perturbation test (arXiv:2604.15149) on the selected candidate: re-run with a perturbed seed/split and flag a large drop in the search summary. Cheap and catches memorisation the holdout rank may miss.

Prioritised roadmap

Ordered by evidence strength × implementation cost within the current seams

Prio	Change	Why first / evidence
1	O1 error-fix loop inside the operator call (+ E4 stage-1 smoke verifier)	PIKE: dedicated error fixing was the single largest gain; also cuts cost per valid candidate
1	P1 configurable operator rule (debug prob, depth, improve-failure cap per branch)	Small change; stops the greedy policy from re-improving a plateau (ML-Master τ_{improve})
1	E2 gates next to the score	Cheapest reward-hacking defence; unlocks leakage/data-usage checks (MLE-STAR)
2	O2 hypothesis / learnings fields per candidate	Improves sibling context and makes claims distillation deterministic (R&D-Agent, pi)
2	P3 + O3 stagnation triggers + fusion operator	MLEvolve ablation: −13 pts without cross-branch recombination
2	E1 verifier isolation (separate venv, read-only candidate, private dir)	CORAL; needed before exposing hillclimb to untrusted problems or external users
2	P5 novelty gate before agent call	Shinka; each sample is expensive, duplicates are common after many improves
3	P2 UCT policy; P4 exploit preset for OpenEvolve	AIRA: only pays once operators are strong; preset is a config change
3	O5 cost-aware bandit; E3 harden CLI; E5/E6 metric allow-list, winner re-scoring	Valuable but each independent; schedule after the above
3	P6 cross-search merge; O6 meta-prompt evolution; remote execution pool	Larger surface; revisit when multi-machine runs or open-ended problems become the norm

What not to do: invest in exotic search policies before O1–O3. Both controlled studies in the list (AIRA, PIKE) found that operator quality and error repair, not the policy, set the ceiling. hillclimb's seam makes policy work cheap later, so the order above is safe.

Sources

Read for this deck (22 Aug 2026)

Index

- [../awesome-auto-hillclimb/README.md](#) (key findings, project tables, reward-hacking section)

hillclimb

- `src/hillclimb/` : `search.py`, `policy.py`, `policies/{greedy,openevolve}.py`, `routing.py`, `bandit.py`, `backends/claude_code.py`, `executor.py`, `problem.py`, `baseline.py`, `journal.py`, `knowledge.py`, `claims.py`, `graph.py`, `credit.py`, `consolidate.py`, `skills.py`; `README.md`; `CLAUDE.md`;
• `.claude/skills/hillclimb/SKILL.md`

ML-engineering agents

- AIDE: github.com/WecoAI/aideml; arXiv:2502.13138
- AIRA-dojo: github.com/facebookresearch/aira-dojo; arXiv:2507.02554
- weco-cli: github.com/WecoAI/weco-cli
- R&D-Agent: github.com/microsoft/RD-Agent; arXiv:2505.14738
- MLE-STAR: arXiv:2506.15692 (ADK sample path 404)
- MLEvolve: github.com/InternScience/MLEvolve; arXiv:2606.06473
- ML-Master: arXiv:2506.16499

Evolutionary program search

- OpenEvolve 0.3.2 local source (.venv site-packages) + github.com/algorithmicsuperintelligence/openevolve
- ShinkaEvolve: github.com/SakanaAI/ShinkaEvolve; arXiv:2509.19349
- CodeEvolve: github.com/inter-co/science-codeevolve; arXiv:2510.14150
- FM Agent: github.com/baidubce/FM-Agent (engine closed); arXiv:2510.26144
- PIKE: github.com/pike-project/pike, [pike-openevolve](https://github.com/pike-openevolve); arXiv:2511.16964
- AlphaEvolve: arXiv:2506.13131; FunSearch: github.com/google-deepmind/funsearch

Autoresearch loops

- [karpathy/autoresearch](https://github.com/karpathy/autoresearch) (`program.md`, `train.py`)
- pi-autoresearch: github.com/davebcn87/pi-autoresearch
- evo: github.com/evo-hq/evo
- CORAL: github.com/Human-Agent-Society/CORAL; arXiv:2604.01658

Evaluator integrity

- arXiv:2606.08960 (hacker/fixer loops), 2604.15149 (isomorphic perturbation), 2605.21384 (SpecBench)

Caveats: several GitHub doc pages were 404 at read time (noted per system); benchmark numbers are as self-reported by each project; FM Agent internals are from the paper only.